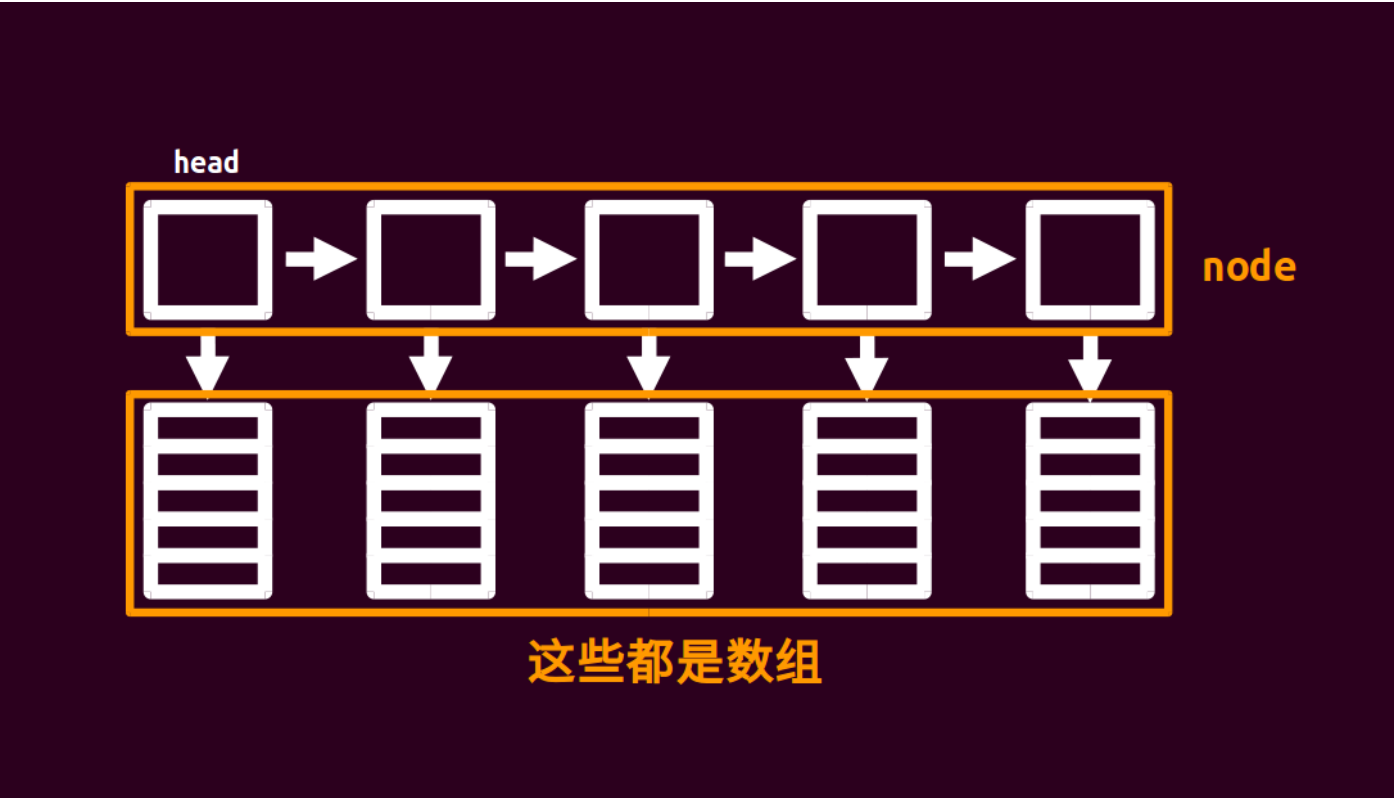


Block List

块状链表



块状链表大概就长这样.....

不难发现块状链表就是一个链表，每个节点指向一个数组。我们把原来长度为 n 的数组分为 $\sqrt{n}$ 个节点，每个节点对应的数组大小为 $\sqrt{n}$ 。所以我们这么定义结构体，代码见下。其中 sqn 表示 $\sqrt{n}$ 即， pb 表示 push_back ，即在这个 node 中加入一个元素。

▼ 实现

```
1 struct node {
2     node* nxt;
3     int size;
4     char d[(sqn << 1) + 5];
5
6     node() { size = 0, nxt = NULL, memset(d, 0, sizeof(d)); }
7
8     void pb(char c) { d[size++] = c; }
9 };
```

块状链表应该至少支持：分裂、插入、查找。什么是分裂？分裂就是分裂一个 node ，变成两个小的 node ，以保证每个 node 的大小都接近 $\sqrt{n}$ （否则可能退化成普通数组）。当一个 node 的大小超过 $2\sqrt{n}$ 时执行分裂操作。

分裂操作怎么做呢？先新建一个节点，再把被分裂的节点的后 $\sqrt{n}$ 个值 copy 到新节点，然后把被分裂的节点的后 $\sqrt{n}$ 个值删掉 (size--)，最后把新节点插入到被分裂节点的后面即可。

块状链表的所有操作的复杂度都是 $O(\sqrt{n})$ 的。

还有一个要说的。随着元素的插入（或删除）， $\sqrt{n}$ 会变， n 也会变。这样块的大小就会变化，我们难道还要每次维护块的大小？

其实不然，把 $\sqrt{n}$ 设置为一个定值即可。比如题目给的范围是 $[1, 10^6]$ ，那么 $\sqrt{n}$ 就设置为大小为 1000 的常量，不用更改它。

```
1 list<vector<char>> orz_list;
```

STL 中的 rope

导入

STL 中的 `rope` 也起到块状链表的作用，它采用可持久化平衡树实现，可完成随机访问和插入、删除元素的操作。

由于 `rope` 并不是真正的用块状链表来实现，所以它的时间复杂度并不等同于块状链表，而是相当于可持久化平衡树的复杂度（即）。

可以使用如下方法来引入：

```
1 #include <LText/rope>
2 using namespace __gnu_cxx;
```

▼ 关于双下划线开头的库函数

OI 中，关于能否使用双下划线开头的库函数曾经一直不确定，2021 年 CCF 发布的 [关于 NOI 系列活动中编程语言使用限制的补充说明](#) 中提到「允许使用以下划线开头的库函数或宏，但具有明确禁止操作的库函数和宏除外」。故 `rope` 目前可以在 OI 中正常使用。

基本操作

操作	作用
<code>rope <LTint > a</code>	初始化 <code>rope</code> （与 <code>vector</code> 等容器很相似）
<code>a.push_back(x)</code>	在 <code>a</code> 的末尾添加元素 <code>x</code>
<code>a.insert(pos, x)</code>	在 <code>a</code> 的 <code>pos</code> 个位置添加元素 <code>x</code>
<code>a.erase(pos, x)</code>	在 <code>a</code> 的 <code>pos</code> 个位置删除 <code>x</code> 个元素
<code>a.at(x)</code> 或 <code>a[x]</code>	访问 <code>a</code> 的第 <code>x</code> 个元素
<code>a.length()</code> 或 <code>a.size()</code>	获取 <code>a</code> 的大小

例题

[POJ2887 Big String](#)

题解： 很简单的模板题。代码如下：

```

1 #include <Ltcctype>
2 #include <Ltcstring>
3 #include <LTiostream>
4 using namespace std;
5 constexpr int sqn = 1e3;
6
7 struct node { // 定义块状链表
8     node* nxt;
9     int size;
10    char d[(sqn << 1) + 5];
11
12    node() { size = 0, nxt = NULL; }
13
14    void pb(char c) { d[size++] = c; }
15 }* head = NULL;
16
17 char inits[(int)1e6 + 5];
18 int llen, q;
19
20 void readch(char& ch) { // 读入字符
21     do cin >> ch;
22     while (!isalpha(ch));
23 }
24
25 void check(node* p) { // 判断, 记得要分裂
26     if (p->size >= (sqn << 1)) {
27         node* q = new node;
28         for (int i = sqn; i < p->size; i++) q->pb(p->d[i]);
29         p->size = sqn, q->nxt = p->nxt, p->nxt = q;
30     }
31 }
32
33 void insert(char c, int pos) { // 元素插入, 借助链表来理解
34     node* p = head;
35     int tot, cnt;
36     if (pos > llen++) {
37         while (p->nxt != NULL) p = p->nxt;
38         p->pb(c), check(p);
39         return;
40     }
41     for (tot = head->size; p != NULL && tot < pos; p = p->nxt, tot += p->size);
42     tot -= p->size, cnt = pos - tot - 1;
43     for (int i = p->size - 1; i >= cnt; i--) p->d[i + 1] = p->d[i];
44     p->d[cnt] = c, p->size++;
45     check(p);
46 }
47
48 char query(int pos) { // 查询
49     node* p;
50     int tot;
51     for (p = head, tot = head->size; p != NULL && tot < pos;
52          p = p->nxt, tot += p->size);
53     tot -= p->size;
54     return p->d[pos - tot - 1];
55 }
56
57 int main() {
58     cin.tie(nullptr)->sync_with_stdio(false);
59     cin >> inits >> q;
60     llen = strlen(inits);
61     node* p = new node;
62     head = p;
63     for (int i = 0; i < llen; i++) {
64         if (i % sqn == 0 && i) p->nxt = new node, p = p->nxt;
65         p->pb(inits[i]);
66     }
67     char a;
68     int k;
69     while (q--) {
70         readch(a);
71         if (a == 'Q')
72             cin >> k, cout << query(k) << '\n';
73         else
74             readch(a), cin >> k, insert(a, k);
75     }
76     return 0;
77 }

```

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [HeRaNO](#), [konnyakuxzy](#), [Chrogeek](#), [ChungZH](#), [Enter-tainer](#), [iamtwz](#), [lr1d](#), [kenlig](#), [ksyx](#), [littlefrog](#), [littlefrogfromthenorth](#), [megakite](#), [shuzhouliu](#), [StudyingFather](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用